overlapping block Spiking Transformer with Spatial-Temporal Attention (obSTAtten)

Brandon Esebag

1. Introduction

Link to GitHub Repository

Contains a readme file, LaTeX code, project code, this presentation and all trained data used in the paper.

This repository should be publicly available for the foreseeable future.

Summary of Introduction

- Spiked Neural Networks used in tandem with neuromorphic hardware are very energy efficient due to sparsity of internal signals
- Neuromorphic computing consists of neuromorphic hardware, sensors and algorithms, and when used together this enables low energy control for UAVs, robotics, and biomedical imaging devices
- **This Paper's Contribution:**
    - This paper proposes obSTAtten, a modified spatial-temporal attention model for spiking transformers that supports overlapping temporal blocks.
    - This paper evaluates obSTAtten against STAtten and a `default attention model' over T=4 static inputs.

2. Related Works

Spiking Neural Networks

- Spiking Neural Networks are inspired by real neurons (more so than ANN neurons)
- Spiking Neurons communicate in pulses (shown to the right)
- There are many Spiking Neuron models, the two that were used to test obSTAtten were LIF and PLIF neurons

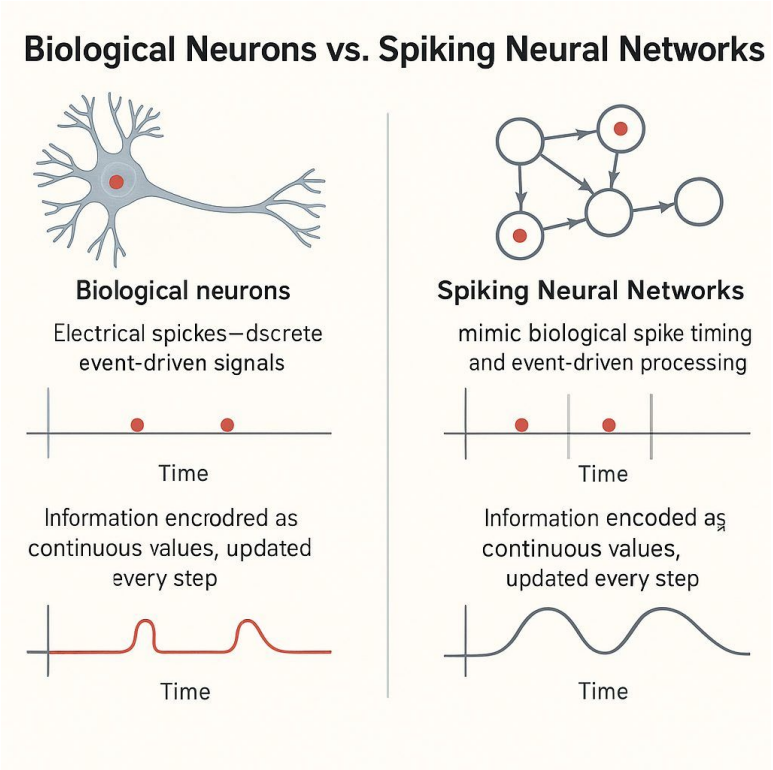

Image Source:
https://aicompetence.org/spiking-neural-networks-on-fpga-speed-efficiency/

Leaky Integrate-and-Fire Neuron Model

T: Timestep

V(t) / u[t]: Current Membrane Potential

V_{rest}: Resting Membrane Potential

X(t): Current Neuron Input

τ: Membrane Time Potential (Leak Rate)

W: Weight

l: Referenced Layer

$$\tau \frac{dV(t)}{dt} = -(V(t) - V_{rest}) + X(t) \tag{1}$$

$$u[t+1]^l = \tau u[t]^l + W^l f(u[t]^{l-1}) \tag{2}$$

$$f(u[t]^l) = \begin{cases} 1, & \text{if } u[t]^l > V_{th} \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

Parametric LIF Neuron

- The parameter τ is no longer a hyperparameter and is treated as a trainable parameter
- τ is typically shared across an entire layer
- This gives the network better temporal response than typical LIF models
- PLIF neurons are defined by the three behaviors shown to the right

1. The time constant τ is automatically set during model training
2. The membrane time constant τ is shared within each layer of the model, which is biologically plausible (and is conveniently compatible with the activation function shown in (2) and (3))
3. The membrane time constant τ is not shared between layers, allowing diverse phase-frequency responsiveness

Vision Transformer (ViT)

- When an image is fed into the model, the ViT splits it into P patches.
- Different heads that train to correlate for Q, K, and V (Query, Key, Value)
- Transformers allow non-contiguous regions of input data to be contextually correlated
- Correlation is done using an attention model

An image $x \in \mathbb{R}^{W \times H \times C}$ of width W, height H and C channels is input into the model and split into P patches. This generates N tokens (4), allowing each patch to be flattened into a trainable embedding vector (5)[8]. From this $x_p^i \in \mathbb{R}^{(P^2C)}$ is the i-th patch, $E \in \mathbb{R}^{(P^2C) \times D}$ is a learnable projection matrix and the matrix E_{pos} denotes positional embeddings[8].

$$N = \frac{WH}{P^2} \tag{4}$$

$$z_0 = [x_p^1\mathbf{E}; x_p^2\mathbf{E}; ...x_p^N\mathbf{E};] + E_{pos} \tag{5}$$

Standard Self Attention

This input spike sequence can be used to generate three projections; $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ which represent the Query, Key and Value[8] projections of the input. For a floating-point input tensor X_f with N tokens and D features (6), the floating point attention projections $\mathbf{Q_f}$, $\mathbf{K_f}$ and $\mathbf{V_f}$ can be found using the trained weights $\mathbf{W_Q}$, $\mathbf{W_K}$ and $\mathbf{W_V}$ (7) where $\mathbf{W_Q}, \mathbf{W_K}, \mathbf{W_V} \in \mathbb{R}^{D \times D}$.

$$X_f \in \mathbb{R}^{N \times D} \tag{6}$$

$$Q_f = W_Q X_f, \quad K_f = W_K X_f, \quad V_f = W_V X_f \tag{7}$$

Using the floating point projections $\mathbf{Q_f}$, $\mathbf{K_f}$ and $\mathbf{V_f}$, standard self attention can be modeled as shown in (8)[8]. Notice that this attention model does not have a dimension of time and requires floating point input. Without modification this cannot be used for spiking neural networks.

$$\text{Atn} = \text{Softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{D}}\right) V, \quad Q_f, K_f, V_f \in \mathbb{R}^{N \times D} \tag{8}$$

Spiking Self Attention

- Spiking Self Attention (SSA) here is defined for an LIF neuron of arbitrary τ
- This attention model can be tested with both LIF and PLIF neurons

Spiking neural networks cannot directly process floating point input data such as X_f, thus for a binary tensor $X \in \mathbb{R}^{\mathtt{T} \times \mathtt{N} \times \mathtt{D}}$ for timestep T, spiking self attention (SSA) can be found via (10) for $\mathbf{Q}, \mathbf{K}$ and $\mathbf{V}$ (9). Note that α is a scaling factor and is not the learning rate (as it would be for an ANN).

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathrm{LIF}(\mathbf{W_Q X}), \mathrm{LIF}(\mathbf{W_K X}), \mathrm{LIF}(\mathbf{W_V X}) \quad (9)$$

$$\mathrm{SSA}(\mathbf{QKV}) = \mathrm{LIF}(\mathbf{QK}^T\mathbf{V} \cdot \alpha) \quad (10)$$

Spiking Transformer with Spatial-Temporal Attention

$$\text{STAtten}(X[b]) = \text{LIF}(\mathbf{Q}[b]\mathbf{K}^T[b]\mathbf{V}[b] \cdot \alpha) \tag{11}$$

$$[b] = [iB : (i+1)B, :, d], \quad i \in \{0, 1, ..., T/B - 1\} \tag{12}$$

The notation $[iB : (i+1)B, :, d]$ represents the aggregated features from timesteps iB to $(i+1)B$[6]. As an example, for $T = 4$ and $B = 2$, this will create blocks $[0, 1]$ and $[2, 3]$ where the features contained within paired timesteps are compared while other features are not, making computation manageable[6]

First, we define the three different types of attention: Spatial, Temporal, and Spatial-temporal attention with timestep t, token position n, and feature dimension d as follows:

$$\begin{aligned}
\text{S_Attn}[t, :, :] &= \text{LIF}(\mathbf{Q}[t, :, :]\mathbf{K}^\top[t, :, :]\mathbf{V}[t, :, :] \cdot \alpha), \\
\text{T_Attn}[:, n, :] &= \text{LIF}(\mathbf{Q}[:, n, :]\mathbf{K}^\top[:, n, :]\mathbf{V}[:, n, :] \cdot \alpha), \\
\text{ST_Attn}[:, :, d] &= \text{LIF}(\mathbf{Q}[:, :, d]\mathbf{K}^\top[:, :, d]\mathbf{V}[:, :, d] \cdot \alpha),
\end{aligned} \tag{6}$$

where S_Attn, T_Attn, and ST_Attn are spatial, temporal, and spatial-temporal attention, and the $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ dimensions of S_Attn, T_Attn, and ST_Attn are $\mathbb{R}^{T \times N \times D}$.

Spiking Transformer with Spatial-Temporal Attention

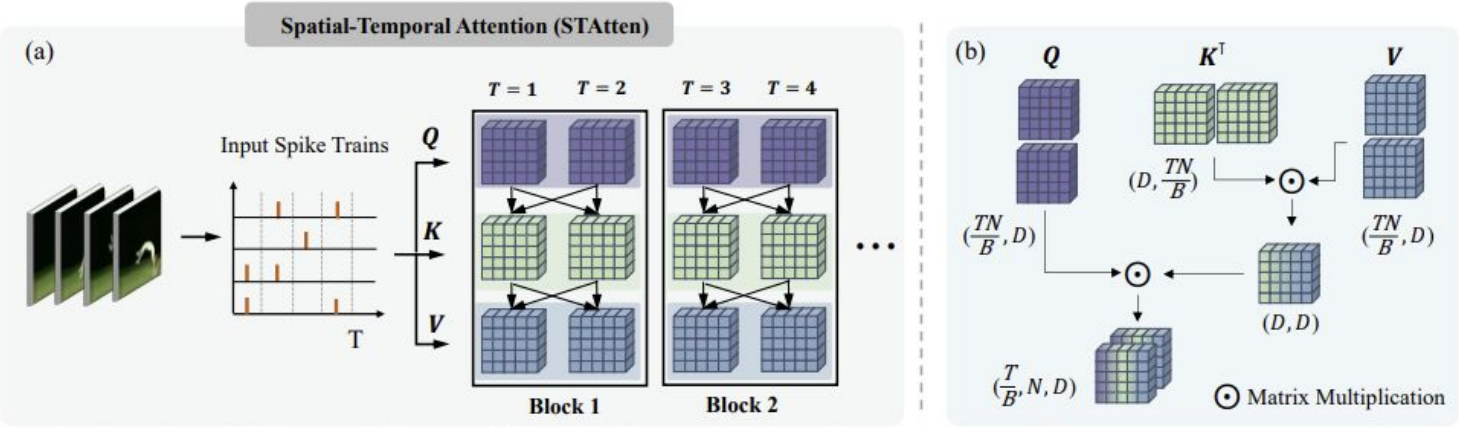

Figure 4. Overview of STAtten architecture. **(a)** Block-wise temporal attention mechanism. Binary Q, K, V tensors are partitioned into temporal blocks, where black lines indicate paired timestep processing. **(b)** Computation flow with tensor dimensions, where T is the number of timesteps, N is the number of tokens, B is block size, and D is the feature dimension.

3. Proposed Approach

obSTAtten Architecture Concept

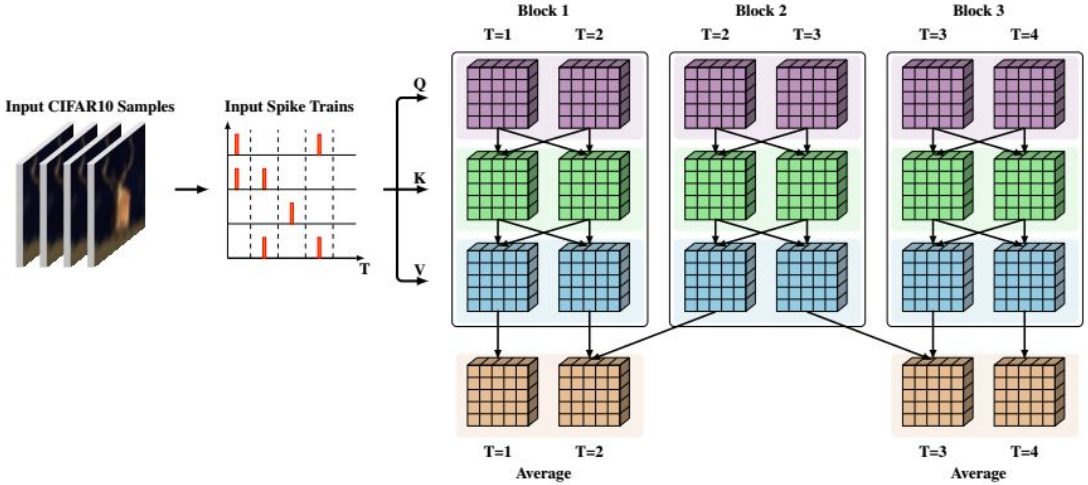

Figure 1. obSTAtten Attention Model showing determined block count and temporal output averaging

obSTAtten Change 1: Determining Block Count

In obSTAtten the number of temporal blocks N_B is slightly more complex to calculate. This complexity results from the allowance of overlapping temporal blocks which no longer create non-overlapping temporal partitions. For simplicity the model has been configured for an overlap of $T_{ov} = 1$ and a stride $s = 1$, resulting in the relationship shown in (16). As before, N_B is the number of blocks, T is the number of time-samples t and S_B is the block size.

$$N_B = T - S_B + 1 \tag{16}$$

obSTAtten Change 1: Determining Block Count

```
## obSTAtten Attention Mode (newly added) -- "overlapping block STAtten"
if self.attention_mode == "obSTAtten":
    if self.dvs:
        scaling_factor = 1 / (H * H * self.chunk_size)
    else:
        scaling_factor = 1 / H

    # This model produces number blocks that overlap by 1, there should be T - chunk_size + 1 chunks
    num_chunks = T - self.chunk_size + 1
```

obSTAtten Change 2: Overlapping Block Construction

For T time samples and N_B blocks, overlapping block $\mathcal{B}_i$ is formed by (17) for index i. Note that if the index i had a stride $s = S_B$ this would output the same set of blocks as STAtten.

$$\mathcal{B}_i = [i : i = N_B, :, d], \quad i \in 0, 1, ..., T - S_B \tag{17}$$

obSTAtten Change 2: Overlapping Block Construction

```
# Replace .view() with .unfold() as .view() only works for contiguous/non-overlapping memory
# .unfold() needs dimension, size and step to slide a window over a dimension
q_chunks = q.unfold(0, self.chunk_size, 1)
k_chunks = k.unfold(0, self.chunk_size, 1)
v_chunks = v.unfold(0, self.chunk_size, 1)
# shape: (num_chunks, B, num_heads, N, head_dim, chunk_size)

# STAtten permutes (num_chunks, chunk_size, B, num_heads, N, head_dim) to (num_chunks, B, num_heads, chunk_size, N, head_dim)
# .unfold() puts chunk_size at the END so we move it to position 3, the new data must be rearranged to match
q_chunks = q_chunks.permute(0, 1, 2, 5, 3, 4)
k_chunks = k_chunks.permute(0, 1, 2, 5, 3, 4)
v_chunks = v_chunks.permute(0, 1, 2, 5, 3, 4)
# shape: (num_chunks, B, num_heads, chunk_size, N, head_dim)
```

obSTAtten Change 3: Timestep Averaging

Let the final model output at timestep t be $\hat{Y}_t$ where $i : t \in \mathcal{B}_i$ is the set of overlapping blocks containing timestep t. Since multiple blocks are contributing to $\hat{Y}_t$ their contributions must be averaged. This is done via (19).

$$\hat{Y}_t = \frac{1}{|i : t \in \mathcal{B}_i|} \sum_{i:t \in \mathcal{B}_i} \text{obSTAtten}(X_{\mathcal{B}_i})_t \tag{19}$$

Note that timesteps $t = 0$ and $t = T - 1$ are not averaged because they each only appear in one block. This behavior is shown in Figure 1.

obSTAtten Change 3: Timestep Averaging

```
# Unlike in STAtten where each time-step is unique to one chunk, obSTAtten has overlap
# The output from each time-step should be the average of the two places it appears
output_acc = torch.zeros(T, B, self.num_heads, N, head_dim,
                         device=x.device, dtype=out.dtype)
count = torch.zeros(T, device=x.device, dtype=out.dtype)

for i in range(num_chunks):
    # chunk i covers timesteps [i, i+1, ..., i+chunk_size-1]
    output_acc[i : i + self.chunk_size] += out[i].permute(3, 0, 1, 2, 4)
    count[i : i + self.chunk_size] += 1

# Divide each timestep by how many chunks contributed to it
# count shape: (T,) -> reshape to (T,1,1,1,1) to broadcast
output = output_acc / count.reshape(T, 1, 1, 1, 1)
# shape: (T, B, num_heads, N, head_dim) is same as STAtten output
```

4. Experiment

Running the Training Pipeline

Spike Modes: lif, plif

Attention Modes: SDT, STAtten, obSTAtten

Dimension Sizes: 256, 512

(see -c conf/cifar10/2_512_200E_t4.yml \ )

Command Example:

```
torchrun --nproc_per_node=1 train.py \
-c conf/cifar10/2_512_200E_t4.yml \
--model sdt \
--spike-mode lif \
--attention_mode STAtten \
--batch-size 4
```

Limitations

Hardware:

- GTX 1070 TI GPU (very old)
  - Limits batch size to 4
- ~100 GB of HDD space on the system (not enough to run CIFAR10-DVS)
  - Would sometimes run out of space mid-run

Time:

Each run takes ~20-30 hours, this means I could only do 1-2 runs each 48 hours. I did not have time to make any more data, however I managed to test 12 scenarios over the course of this project.

5. Results

Tabular Results

Table 1. Dim-256 LIF Results

Attention	SDT	STAtten	obSTATTen
Last Top-1	93.08%	**93.13%**	**93.13%**
Best Top-1	**93.33%**	93.30%	93.30%
Best Epoch	205	**199**	**199**
Training Time	17.10 hrs	**16.99 hrs**	17.26 hrs

Table 2. Dim-256 PLIF Results

Attention	SDT	STAtten	obSTATTen
Last Top-1	**93.44%**	93.32%	93.41%
Best Top-1	**93.61%**	93.32%	93.48%
Best Epoch	**204**	209	209
Training Time	**18.84 hrs**	18.96 hrs	19.33 hrs

Table 3. Dim-512 LIF Results

Attention	SDT	STAtten	obSTATTen
Last Top-1	**94.42%**	**94.42%**	94.28%
Best Top-1	94.50%	94.50%	**94.76%**
Best Epoch	208	208	**206**
Training Time	28.74 hrs	28.47	**28.30 hrs**

Table 4. Dim-512 PLIF Results

Attention	SDT	STAtten	obSTATTen
Last Top-1	94.29%	**94.62%**	93.26%
Best Top-1	94.57%	**94.63%**	93.34%
Best Epoch	200	208	**194**
Training Time	**30.34 hrs**	30.47 hrs	30.67 hrs

512-dim, LIF, Training Times per Epoch

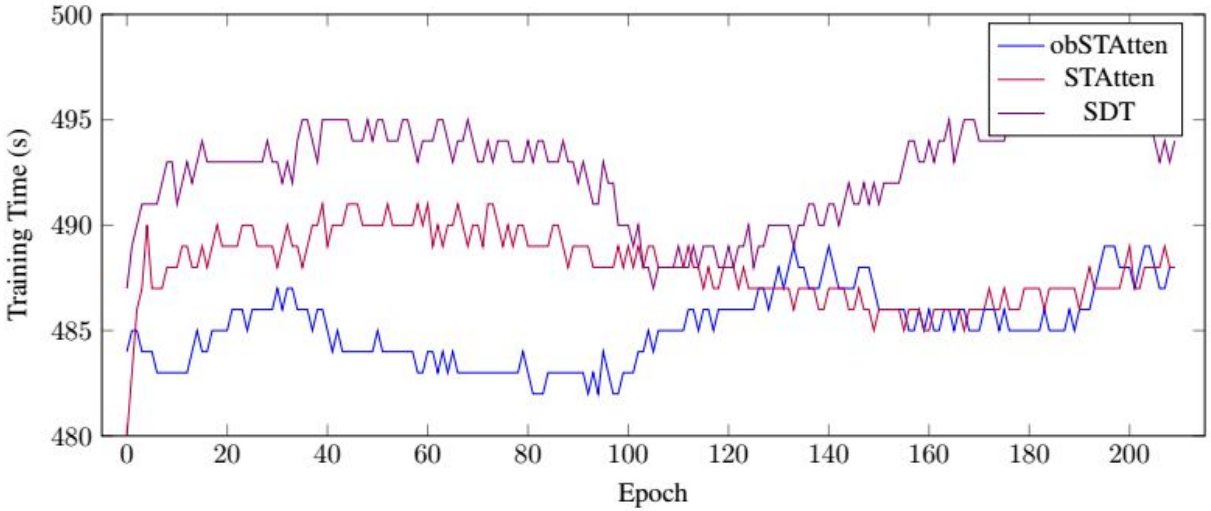

Figure 2. 512-dim, LIF Model train time on CIFAR-10

256-dim, LIF, STAtten obSTAtten Convergence

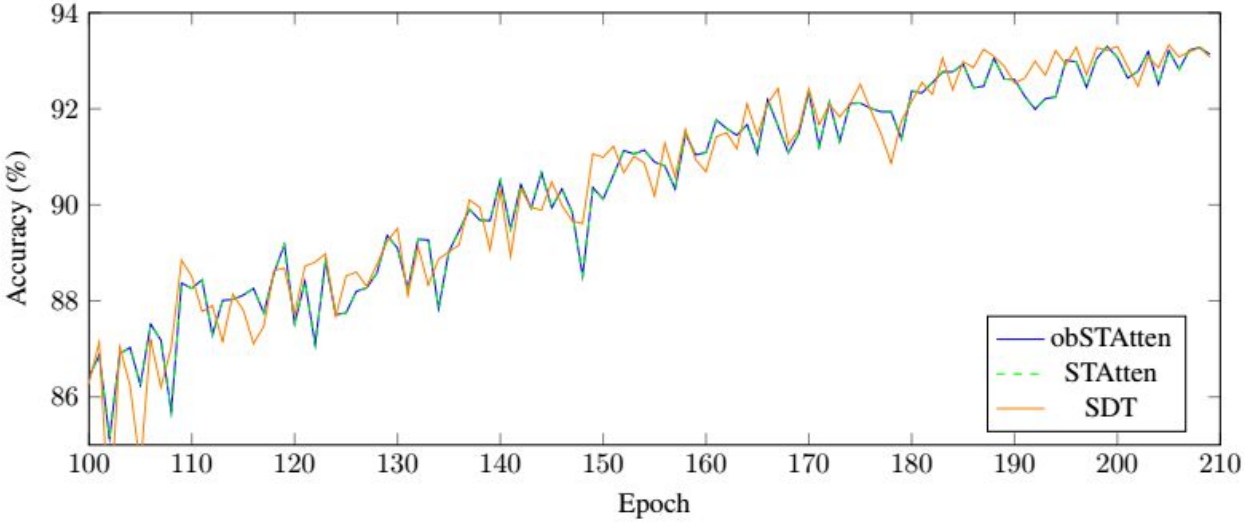

Figure 3. 256-dim, LIF Model Test accuracy on CIFAR-10

Other Observations

LIF vs PLIF on obSTAtten Performance

- Both STAtten and obSTAtten perform worse relative to SDT for PLIF
- The extra temporal information may confuse PLIF (which already has temporal information)
- The extra temporal information seems to help LIF (which does not have as much temporal information)

STAtten vs obSTAtten for Best-Epoch

- In both dim-512 cases obSTAtten reaches its peak faster than STAtten does
- This happens both for LIF (when obSTAtten does better) and PLIF (when STAtten does better)

5. Conclusion

Conclusion

Limitations and Shortcomings

The paper only explored the effect of CIFAR10 with static images, which likely limited obSTAtten's ability to express unique some behaviors

SNNs are inherently temporal and I would like to continue experimenting with this using DVS

obSTAtten only conditionally outperforms STAtten, but it does (in some cases) outperform STAtten

Future Work

The paper proposes two directions for future work depending on hardware constraints:

- Explore more stride and overlap options for CIFAR10 and similar settings
- Invest in better equipment and study the effects of DVS on the current obSTatten

Questions?